



[Home](#)
[Interview Questions](#)
[Java Programs](#)
[Pyramid Programs](#)
[Programming Challenges](#)
[Download Notes](#)
[What's new..?](#)

Servlet interview Questions - 3

1- What are the new features added to Servlet 2.5?

Following are the new features introduced in Servlet 2.5:

- A new dependency on Java SE 5.0
- Support for annotations
- Loading the class
- Several web.xml conveniences
- A handful of removed restrictions
- Some edge case clarifications

2- Why do we need a constructor in a servlet if we use the init method?

Even though there is an init method in a servlet which gets called to initialize it, a constructor is still required to instantiate the servlet. Even though you as the developer would never need to explicitly call the servlet's constructor, it is still being used by the container (the container still uses the constructor to create an instance of the servlet).

Let us say in a plain java class, you created an init method that gets invoked under some condition, will you be able to invoke it without actually instantiating your class? That is why we need a constructor even for a Servlet.

3- When is the servlet loaded?

A servlet can be loaded when:

First request is made.

Server starts up (auto-load).

There is only a single instance which answers all requests concurrently. This saves memory and allows a Servlet to easily manage persistent data.

Administrator manually loads.

4- When is a Servlet unloaded?

A servlet is unloaded when:

Server shuts down.

Administrator manually unloads.

5- What is the GenericServlet class?

GenericServlet is an abstract class that implements the Servlet interface and the ServletConfig interface. In addition to the methods declared in these two interfaces, this class also provides simple versions of the lifecycle methods init and destroy, and implements the log method declared in the ServletContext interface.

Like 12k

G+1

2.2k

5 Online Now



Search Program

Follow by Email

-59%

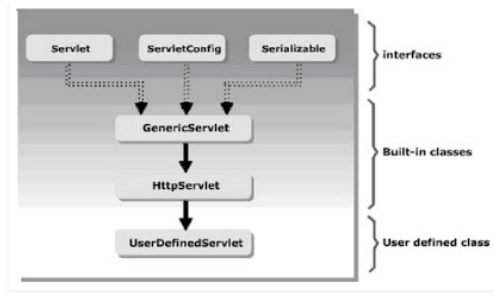
SHOP NOW

-52%

SHOP NOW

Labels

[Addition of number](#)
[Advance Java](#)
[Android](#)
[AP Series](#)
[Armstrong](#)
[Number Arrays](#)
[assertion](#)
[Binary](#)
[Addition](#)
[binary](#)
[Multiplication](#)
[Binary to Decimal](#)
[Binary to Hexadecimal](#)
[C](#)
[C++](#)
[Class](#)
[Coding](#)
[Standard](#)
[Collection](#)
[Conversion](#)
[Core Java](#)
[Daemon](#)
[Thread](#)
[Datatypes](#)
[Decimal To Binary](#)
[Decimal to hexadecimal](#)
[Decimal to octal](#)
[Diamond](#)
[Durga](#)
[Sir Durgasoft](#)
[Encapsulation](#)
[Exception](#)
[Handling](#)
[Factorial](#)
[Features](#)
[Fibonacci](#)
[Garbage](#)
[Collection](#)
[Generic](#)
[Root](#)
[GP](#)
[Series](#)
[HCF](#)
[and](#)
[LCF](#)
[Hibernate](#)
[HP](#)
[Series](#)
[HR](#)
[Questions](#)
[BM](#)
[DB-2](#)
[IBPS](#)
[Identifiers](#)
[Indore](#)
[Interface](#)
[Interview](#)
[Interview](#)
[Question](#)
[Java](#)



[Java](#) [Program](#) [Java4732](#) [JavaEE](#) [JavaSE](#)
[JDBC](#) [JSP](#) [Linux](#) [Literals](#) [Matrix](#) [Multi](#) [threading](#) [Natraj](#) [sir](#)
[Natraj](#) [Technology](#) [NCR](#) [Notes](#) [Number](#) [Pattern](#) [Number](#)
[System](#) [Object](#) [Octal](#) [to](#) [binary](#) [Octal](#) [to](#) [Decimal](#) [OOPS](#) [Oracle](#)
[Palindrome](#) [Pattern](#) [PDF](#) [Polymorphism](#) [Practice](#) [Programs](#)
[Prime](#) [Number](#) [Programming](#) [Challenges](#)
[Programs](#) [Pyramid](#) [Real-Life-Example](#) [Rectangle](#)
[Pattern](#) [Recursion](#) [Reserved](#) [Words](#) [Reverse](#) [Number](#) [SAP](#) [SCP](#)
[Serialization](#) [Series](#) [Series](#) [Servlet](#) [Session](#) [Silver](#) [Light](#)
[Software](#) [Companies](#) [Spring](#) [Star](#) [Pattern](#) [String](#) [Struts-2](#) [Sum](#) [of](#)
[digits](#) [Swapping](#) [Two](#) [Array](#) [Top](#) [10](#) [triangle](#) [tutorial](#) [var-args](#)
[Method](#) [Video](#) [Tutorial](#) [Web](#) [Logic](#) [Web](#) [Service](#) [Written](#)
[Test](#)

Note: This class is known as generic servlet, since it is not specific to any protocol whereas the HttpServlet is specific to the Http Protocol

6~ Why is the HttpServlet class declared abstract?

The HttpServlet class is declared abstract because the default implementations of the main service methods do nothing and must be overridden. Just like any other abstract class, the HttpServlet is a partial skeleton which defines basic behavior and then gives the programmer the freedom to implement any of the other possible features he/she wants.

The simplest example here could be the fact that, any HttpServlet could handle multiple doXXX() methods as we call them which include doGet(), doPost() etc. So, in your applications, if you handle just the POST requests, then you need not implement a doGet() to override the default doGet() implementation.

7~ Can a servlet have a constructor?

Sure Yes. Though we don't explicitly call the constructor using the new() operator, the container internally invokes the Servlet class' container when the servlet is initialized.

8~ Should I override the service() method while using the HttpServlet?

We never override the service method, since the HTTP Servlets have already taken care of it. The default service method invokes the doXXX() method corresponding to the method of the HTTP request. For example, if the HTTP request method is GET, doGet() method is called by default. A servlet should override the doXXX() method for the HTTP methods that servlet supports. Because HTTP service method checks the request method and calls the appropriate handler method, it is not necessary to override the service method itself. Only override the appropriate doXXX() method.

9~ What are the differences between the ServletConfig interface and the ServletContext interface?

Servlet Config Object-

- 1.It is one per our Servlet/JSP program.
- 2.Servlet Config object means it is the object of Servlet container Supplied Java class implement `javax.Servlet.ServletConfig` interface.
- 3.This object is used to pass additional data to Servlet and to

read additional details from Servlet.

- 4.This object is used to read Servlet init() parameter value of the web application.
- 5.Servlet container create the object of Servlet config during instantiation and initialization.
- 6.Servlet container destroys the Servlet config object during destruction process of our Servlet object.

Servlet Context Object-

- 1.It is one per web application so it is called as the global memory of the web application.
- 2.Servlet Context object means it is the object of java class implementing javax.Servlet.ServletContext interface.
- 3.Servlet container create this object during during the deployment of web application.
- 4.Servlet container destroy this object automatically when web application is undeployed or reloaded or Stopped.
- 5.Using this object we can know the detail of underline Server like Server-name,version and Servlet api version.

10- What are the differences between forward() and sendRedirect() methods?

Some key differences between the two methods are:

- a. A Forward is performed internally by the servlet whereas a redirect is a two-step process where the web application instructs the web browser to fetch a second/different URL
- b. The browser and the user is completely unaware that a forward has taken place (The URL Remains intact) whereas in case of redirect, both the browser and the user will be made aware of the action including a change to the URL
- c. The resource to which control is being forwarded has to be part of the same context as the one that is actually calling it whereas in case of redirect this is not a restriction.
- d. Forwards are faster than redirects. Redirects are slower because it is actually handling two browser requests in place of forward's one.

11- What are the difference between the include() and forward() methods?

The key differences between the two methods are:

- a. The include() method inserts the contents of the specified resource directly into the flow of the servlet response, as if it were part of the calling servlet, whereas the forward() is used to show a different resource in place of the servlet that was originally called
- b. The include() is often used to include common text or template markup that may be included in many servlets whereas forward() is often used where a servlet plays the role of a controller, processes some input and decides the outcome by returning a particular page response where control is transferred to a different resource

12- What is the use of the servlet wrapper classes?

The HttpServletRequestWrapper and HttpServletResponseWrapper classes are designed to make it easy for developers to create custom implementations of the servlet request and response types.

The classes are constructed with the standard `HttpServletRequest` and `HttpServletResponse` instances respectively and their default behavior is to pass all method calls directly to the underlying objects.

13- What is a deployment descriptor?

A deployment descriptor is an XML document. It defines a component's deployment settings. It declares transaction attributes and security authorization for an enterprise bean. The information provided by a deployment descriptor is declarative and therefore it can be modified without changing the source code of a bean.

The Java EE server reads the deployment descriptor at run time and acts upon the components accordingly.

14- What is the difference between the `getRequestDispatcher(String path)` method of `javax.servlet.ServletRequest` interface and `javax.servlet.ServletContext` interface?

The `getRequestDispatcher(String path)` method of `javax.servlet.ServletRequest` interface accepts parameter the path to the resource to be included or forwarded to, which can be relative to the request of the calling servlet. If the path begins with a `"/"` it is interpreted as relative to the current context root, whereas, the `getRequestDispatcher(String path)` method of `javax.servlet.ServletContext` interface cannot accept relative paths. All path must start with a `"/"` and are interpreted as relative to current context root.

15- What is the `< load-on-startup >` element of the deployment descriptor?

The element of a deployment descriptor is used to load a servlet file when the server starts instead of waiting for the first request. This setting by which a servlet is loaded even before it gets its first request is called pre-initialization of a servlet. It is also used to specify the order in which the files are to be loaded. The container will load the servlets in the order specified in this element.

16- What is servlet lazy loading?

Lazy servlet loading means - the container does not initialize the servlets as soon as it starts up. Instead it initializes servlets when they receive their first ever request. This is the standard or default behavior. If you want the container to load your servlet at start-up then use pre-initialization using the `load-on-startup` element in the deployment descriptor.

17- What is Servlet Chaining?

Servlet Chaining is a method where the output of one servlet is piped or passed onto a second servlet. The output of the second servlet could be passed on to a third servlet, and so on. The last servlet in the chain returns the output to the Web browser.

18- What are filters?

Filters are Java components that are used to intercept an incoming request to a Web resource or the response that is sent back from the resource. It is used to abstract any useful

information contained in the request or response. Some of the important functions performed by filters are:

Security checks

Modifying the request or response

Data compression

Logging and auditing

Response compression

Filters are configured in the deployment descriptor of a Web application. Hence, a user is not required to recompile anything to change the input or output of the Web application.

19- What are the functions of the Servlet container?

The functions of the Servlet container are as follows:

- 1.Lifecycle management: It manages the life and death of a servlet, such as class loading, instantiation, initialization, service, and making servlet instances eligible for garbage collection.
- 2.Communication support: It handles the communication between the servlet and the Web server.
- 3.Multithreading support: It automatically creates a new thread for every servlet request received. When the Servlet service() method completes, the thread dies.
- 4.Declarative security: It manages the security inside the XML deployment descriptor file.
- 5.JSP support: The container is responsible for converting JSPs to servlets and for maintaining them.



Posted by [Umesh Kushwaha](#)

+13 Recommend this on Google

Labels: [Core Java](#), [Interview Question](#), [Java](#), [Servlet](#)

10 comments



Add a comment as Vijay Kumar

Top comments

**Umesh Kushwaha** · 1 year ago · [Java \(Java EE\)](#)

Servlet interview Questions - Part-3

+5 1



عبدالله العبد الرحمن · 1 year ago · +1

السلام عليكم ورحمة الله وبركاته

· Translate

**Umesh Kushwaha** · 1 year ago

امين

**Umesh Kushwaha** · 1 year ago · [Java \(Java EE\)](#)

Servlet interview Questions - Part 3

1

**Umesh Kushwaha** via Google+ · 1 year ago · Shared publicly**Servlet interview Questions - 3**

1- What are the new features added to Servlet 2.5? Following are the new features introduced in Servlet 2.5: · A new dependency on Java SE 5.0 · Support for annotations · Loading the class · Several web.xml conveniences · A handful of removed restricti...

1 · Reply

**সেলিম মুন্সী** · 1 year ago · Shared publicly

Facebook

1 · Reply

**mickey sanchez** via Google+ · 1 year ago · Shared publicly**Umesh Kushwaha** originally shared this
Servlet interview Questions - Part-3

1

**Claude Martin** · 1 year ago · Shared publicly

Why are there so many duplicates? And what about asynchronous request processing and non-blocking servlets?

1 · Reply

**RANA raghunath** via Google+ · 1 year ago · Shared publicly

great bro

**Umesh Kushwaha** originally shared this
Servlet interview Questions - Part-3

+1 1 · Reply

**Umesh Kushwaha** · 1 year ago
Thanks RANA.[Newer Post](#)[Home](#)[Older Post](#)Subscribe to: [Post Comments \(Atom\)](#)



Copyright © 2016. Simple template. Template images by [PLAINVIEW](#). Powered by [Blogger](#).